Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Grafové editory pre prieskumník štruktúr logiky prvého rádu

Bakalárska práca

2025
Jakub Marček

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Grafové editory pre prieskumník štruktúr logiky prvého rádu

Bakalárska práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Aplikovaná informatika
Školiace pracovisko:	Katedra aplikovanej informatiky
Školiteľ:	Mgr. Ján Kľuka, PhD.

Bratislava, 2025
Jakub Marček

Obsah

Úvod **1**

1 Základné pojmy **3**

1.1 Logika prvého rádu 3

1.2 Grafové reprezentácie 4

1.2.1 Orientovaný graf 4

1.2.2 Hasseho diagram 4

1.2.3 Bipartitný graf 5

1.3 Použité technológie 6

1.3.1 React 6

1.3.2 TypeScript 7

1.3.3 React Flow 7

1.3.4 Redux 7

2 Súvisiace práce **9**

2.1 Prieskumník štruktúr 9

2.2 Prieskumník grafových štruktúr 11

2.3 Logický pracovný zošit 13

3 Návrh **15**

3.1 Požiadavky 15

3.1.1 Zámer rozšírenia aplikácie! 15

3.1.2 Ďalšie požiadavky 16

3.2 Grafové editory 16

3.2.1 Orientovaný graf 16

3.2.2 Bipartitný graf 17

3.2.3 Hasseho diagram 19

3.3 Tabuľkové editory 20

3.3.1 Maticový editor 20

3.3.2 Databázový editor 20

3.4 Editor interpretácií funkcií 20

3.5 Spoločné rozhranie editorov 20
3.6 Nový stav aplikácie 20
3.7 Queries 20

4 Implementácia **21**
4.1 Komponent editorov 21
4.2 Stav aplikácie 21
4.2.1 Štruktúra 21
4.2.2 Synchronizácia 21
4.3 Integrácia s Logickým pracovným zošitom 21
4.3.1 Importovanie a exportovanie stavu 21
4.3.2 Logický kontext 21
4.4 Ďalšie vylepšenia Prieskumníka štruktúr 22
4.4.1 Queries 22
4.4.2 Refaktorizácia Henkinovej-Hintikkovej hry 22

5 Testovanie ? **23**

Záver **25**

Úvod

Možnosti využitia elektronických nástrojov pri výučbe sú v dnešnej dobe široké a oproti konvenčným učebným metódam dokážu poskytnúť interaktívnejší a pútavejší pohľad na skúmanú problematiku. Ich tvorba však predstavuje výzvu, ktorá vyžaduje nielen znalosť danej témy, ale aj schopnosť hľadania kreatívnych a inovatívnych riešení s ňou spojených problémov. Úlohou mojej práce bude vytvorenie prvej verzie takéhoto nástroja, ktorý má slúžiť ako rozšírenie už existujúcich implementácií.

Základ webovej aplikácie, z ktorej táto práca vychádza, vznikol v rámci bakalárskej práce Milana Cifru [5]. Jej úlohou je ponúknuť študentom predmetu Logika pre informatikov interaktívny pohľad na štruktúry pre jazyky logiky prvého rádu. Neskôr sa rozšírením tohto nástroja o prieskumník grafových štruktúr logiky prvého rádu zaoberala bakalárska práca Miroslava Baluchu [4]. Implementácia obsahuje jeden grafový editor, so zámerom poskytnúť študentom alternatívny spôsob vizualizácie a manipulácie týchto štruktúr oproti pôvodnej, menej prehľadnej verzii založenej na textových vstupoch.

Z dôvodu viacerých problémov tejto implementácie sa však ukázala byť medzi študentami málo využívaná. Azda najväčším nedostatkom je snaha samotného grafového editoru nahradiť takmer celú funkcionalitu prieskumníka štruktúr, čo žiaľ na niektorých miestach vedie ku kompromisom znižujúcich jeho prehľadnosť aj intuitívnosť. Ďalším problémom je nedostatok zaujímavých a unikátnych funkcií, v porovnaní s textovými vstupmi, ktoré by ho spravili pre študentov atraktívnejším.

V tejto práci sa pozrieme na spôsoby, akými som sa rozhodol vyriešiť nedostatky predošlej implementácie, najprv navrhnutím a následným implementovaním predstavených riešení. Hlavným cieľom bude poskytnúť študentom rozhranie, ktoré je intuitívne, prehľadné a najmä jednoduché na použitie. Postupne tiež predstavíme niekoľko kľúčových rozdielov. Spomedzi nich najzásadnejším bude využitie troch rôznych grafových reprezentácií (orientovaný graf, bipartitný graf a Hasseho diagram), z ktorých každá dokáže ponúknuť študentovi unikátny pohľad na tú istú štruktúru.

Na úvod zadefinujeme jednotlivé pojmy a technológie používané v ďalších kapitolách. Neskôr sa v prvom návrhu bližšie pozrieme na spôsoby, akými som sa rozhodol dosiahnuť stanovené ciele. Taktiež sa podrobnejšie zameriame na každú grafovú reprezentáciu, aby sme lepšie objasnili hlavnú myšlienku za jej výberom. V závere opíšeme

pilotnú implementáciu a zhodnotíme jej prínosy pre nasledujúce verzie.

Kapitola 1

Základné pojmy

Táto kapitola slúži na oboznámenie čitateľa s pojmami a konceptmi používanými v ďalších častiach práce. Každý pojem sa snaží nielen vysvetliť, ale aj naznačiť motiváciu za jeho výberom, čo má pomôcť k lepšiemu pochopeniu rozhodnutí spravených v ďalších kapitolách.

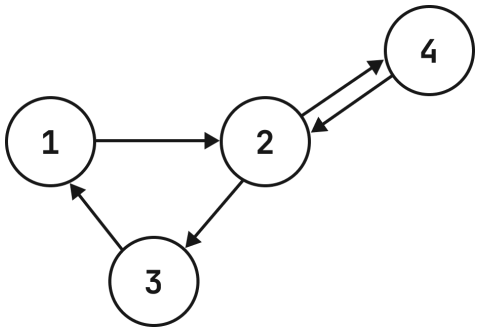
1.1 Logika prvého rádu

Táto podkapitola vychádza najmä z prednášok a poznámok k predmetu Logika pre informatikov [8].

Logika prvého rádu je rodina formálnych jazykov, ktoré hovoria o objektoch a ich vlastnostiach. Symboly konkrétneho jazyka $\mathcal{L}$ logiky prvého rádu tvoria *indivíduové premenné* (napr. x, y, z), *logické symboly* ($\neg$, $\vee$, $\wedge$, $\rightarrow$, $\doteq$, $\exists$, $\forall$), *pomocné symboly* (ľavá zátvorka, pravá zátvorka a čiarka) a *mimologické symboly*, tie sa ďalej delia na *indivíduové konštanty* (napr. Alice, FMFI), *funkčné symboly* (napr. f, g) a *predikátové symboly* (napr. *matka*, *spieva*). Indivíduové premenné, logické symboly a pomocné symboly sú rovnaké naprieč všetkými jazykmi logiky prvého rádu, čo však jednotlivé jazyky odlišuje sú použité mimologické symboly.

Predikátové symboly používame na označenie vlastností alebo vzťahov. V konkrétnom jazyku $\mathcal{L}$ vždy majú (spolu s funkčnými symbolmi) pevne zvolený počet argumentov, ktorý určuje ich *arita*, niekedy označovaná horným indexom. Predikátové symboly s aritou 1 (unárne) zvyknú označovať nejakú vlastnosť alebo stav (napr. $zviera^1, svieti^1$), zatiaľ čo predikátové symboly s aritou 2 a viac (n-árne pre $n \geq 2$) opisujú určitý vzťah svojich argumentov (napr. $stavia^2$, $ukazuje^3$).

Ďalším dôležitým pojmom je *štruktúra*, ktorá sa vždy viaže na nejaký konkrétny jazyk. Označujeme ju dvojicou $\mathcal{M} = (D, i)$, kde D predstavuje ľubovoľnú neprázdnu množinu (doména) a i je zobrazenie (interpretačná funkcia). Úlohou interpretačnej funkcie je každej indivíduovej konštante priradiť prvok z D, každému funkčnému symbolu



Obr. 1.1: Príklad vyobrazenia orientovaného grafu.

priradiť funkciu $f : D^n \to D$ a každému predikátovému symbolu priradiť podmnožinu množiny D^n, kde n v oboch prípadoch označuje aritu daného symbolu. Štruktúry sú dôležitou súčasťou skúmania vlastností jazyka logiky prvého rádu, keďže poskytujú jeho presný matematický model.

1.2 Grafové reprezentácie

V tejto podkapitole sú uvedené typy grafov použité v tejto práci spolu s ich krátkym vysvetlením. Niektoré definície vychádzajú z Grimaldiho učebnice diskrétnej matematiky [7].

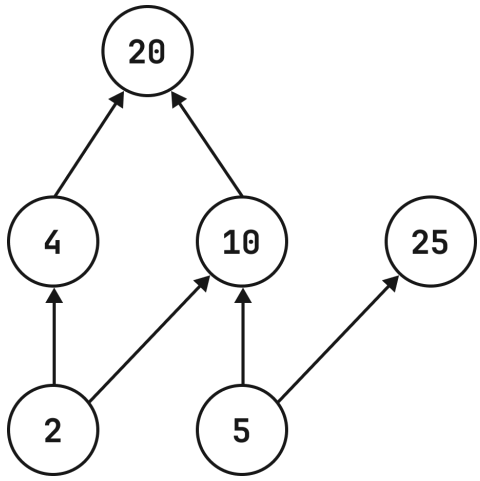
1.2.1 Orientovaný graf

Orientovaný graf G je daný neprázdnou konečnou množinou vrcholov V a množinou usporiadaných dvojíc $E \subseteq V \times V$, resp. hrán, pričom každá hrana $(x, y) \in E$ začína vo vrchole x a končí vo vrchole y. V diagramoch sú vrcholy väčšinou znázornené ako kruh, prípadne spolu s jeho príslušným označením. Hrany sú zas vyobrazené pomocou šípok, ktorých smer určuje poradie vrcholov hrany (šípka vždy ukazuje na druhý vrchol v poradí). Majú mnohoraké využitie pri modelovaní vzťahov, ktoré vyžadujú zachovať nejaký smer, postupnosť úkonov alebo vyjadriť určitú hierarchiu. Príklad bežného diagramu jednoduchého orientovaného grafu možno vidieť na obrázku 1.1, kde $V = \{1, 2, 3, 4\}$ a $E = \{(1, 2), (2, 3), (3, 1), (2, 4), (4, 2)\}$.

1.2.2 Hasseho diagram

Pred opísaním Hasseho diagramu je najprv potrebné objasniť pojem čiastočného usporiadania, ktorý s ním veľmi úzko súvisí.

Podľa definície sa relácia $\leq$ na množine A nazýva *čiastočným usporiadaním*, ak je reflexívna, antisymetrická a tranzitívna. To znamená, že pre každé $x \in A$ musí platiť $x \leq x$ (prvok x je v relácii so sebou samým - reflexívna relácia), pre každé $x, y \in A$



Obr. 1.2: Príklad vyobrazenia Hasseho diagramu.

platí $(x \leq y \wedge y \leq x) \Rightarrow x = y$ (ak je prvok x v relácii s prvkom y a y je v relácii s x, tak $x = y$ - antisymetrická relácia) a pre každé $x, y, z \in A$ platí $(x \leq y \wedge y \leq z) \Rightarrow x \leq z$ (ak je prvok x v relácii s prvkom y a y je v relácii s prvkom z, tak aj x je v relácii so z - tranzitívna relácia). Intuitívnejšie si ale tento pojem možno predstaviť ako dvojice prvkov, z ktorých prvý vždy predchádza druhému. Zvyčajne sa čiastočné usporiadanie označuje aj ako dvojica $(A, \leq)$.

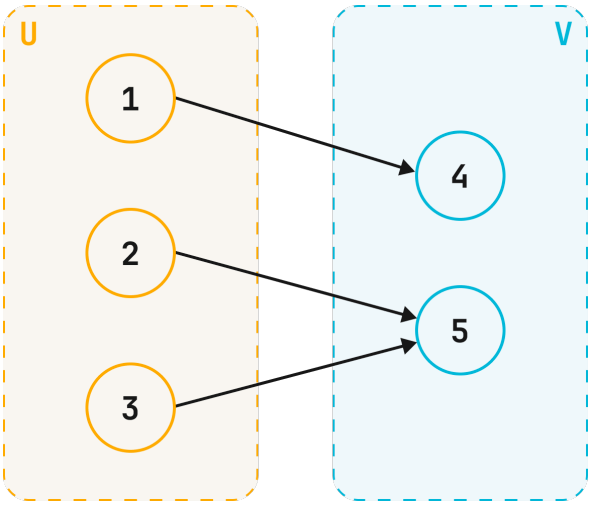
Hasseho diagram slúži na prehľadnejšiu grafickú reprezentáciu čiastočných usporiadaní, keďže zachytáva iba bezprostredné vzťahy, bez zbytočného kreslenia hrán, ktoré intuitívne vyplývajú z jeho vlastností. Pri jeho tvorbe však treba dbať na dodržiavanie určitých pravidiel:

- Každý prvok čiastočného usporiadania, teda prvok množiny A, je vrcholom v takomto grafe.
- Medzi ľubovoľnými prvkami $x, y \in A$ vytvárame hranu iba vtedy, keď $x \leq y$, $x \neq y$ a neexistuje taký prvok $z \in A$, rozdielny od x a y, pre ktorý platí $x \leq z \leq y$.
- Ak je medzi prvkami $x, y \in A$ hrana a v usporiadaní platí $x \leq y$, potom vrchol reprezentujúci prvok x kreslíme pod vrchol reprezentujúci prvok y.

Na obrázku 1.2 možno vidieť príklad Hasseho diagramu pre čiastočné usporiadanie $(A, \mathcal{R})$, kde $A = \{2, 4, 5, 10, 20, 25\}$ a $x \,\mathcal{R}\, y$, ak x delí y bezozvyšku. Dobré je podotknúť, že aj prvky 2 a 5 sú v relácii s prvkom 20, no hrany vyjadrujúce tento vzťah, ako bolo vyššie spomínané, nemusíme do diagramu explicitne zakresľovať.

1.2.3 Bipartitný graf

Bipartitný graf je taký graf G, ktorého vrcholy možno rozdeliť do dvoch disjunktných podmnožín U a V tak, že $U \cap V = \emptyset$ a zároveň každá hrana spája jeden vrchol z U s



Obr. 1.3: Príklad vyobrazenia bipartitného grafu.

ďalším vrcholom z V. Vedia byť nápomocné pri hľadaní a skúmaní vzťahov (relácií a zobrazení) medzi entitami dvoch rôznych tried, ako napríklad medzi študentmi a učiteľmi, autormi a publikáciami alebo receptami a ingredienciami. Pri menšom množstve dát a vhodnom rozložení vrcholov umožňujú prehľadnú vizualizáciu prítomných vzťahov. Ukážku diagramu konkrétneho bipartitného grafu pre zobrazenie $f : \{1, 2, 3\} \to \{4, 5\}$, kde $f(1) = 4, f(2) = 5$ a $f(3) = 5$ so znázorneným rozdelením prvkov do množín U a V (v tomto prípade si možno predstaviť definičný obor a obor hodnôt zobrazenia) je vidieť na obrázku 1.3.

1.3 Použité technológie

V tejto podkapitole sú uvedené kľúčové technológie použité pri pilotnej implementácii.

1.3.1 React

React [9] je v súčasnosti jedna z najpopulárnejších JavaScriptových knižníc na tvorbu responzívnych webových aplikácií.

Základnú stavebnú jednotku tvorí komponent, čo je JavaScriptová funkcia, ktorá produkuje určitý strom elementov jazyka HTML, ďalej označovaný ako markup. Každý takýto komponent potom predstavuje konkrétnu časť používateľského rozhrania s vlastnou funkcionalitou a vzhľadom.

Vo väčšine prípadov sa markup komponentov zapisuje vo forme JavaScript XML (JSX), čo je syntaktické rozšírenie jazyku JavaScript. JSX je veľmi podobný značkovaciemu jazyku HTML, no umožňuje používanie vlastných značiek, ako aj jednoduchšiu manipuláciu s dynamickým obsahom. Toto je pre React kľúčové, keďže je tým umožnené skladať viaceré reaktívne komponenty do určitej hierarchie, ktorá reprezentuje

celú aplikáciu.

1.3.2 TypeScript

TypeScript [10] je programovací jazyk s podporou pre statické typy s konfigurovateľnou mierou typovej kontroly. Funguje iba ako syntaktické rozšírenie JavaScriptu, ktoré sa pri kompilácii naspäť zmení na čistý JavaScript. Práve táto vlastnosť zapríčiňuje jeho širokú kompatibilitu s už existujúcim ekosystémom.

Statické typovanie pomáha odhaľovať chyby už v skorých fázach vývoja, zlepšuje čitateľnosť aj udržateľnosť kódu a vývojárskym prostrediam umožňuje integráciu bohatšej funkcionality, ako napríklad pokročilejšie automatické dopĺňanie na základe kontextu alebo vyhľadávanie definícií. Svojimi prínosmi teda dokáže priamo aj nepriamo uľahčiť a spríjemniť prácu samotného programátora.

Jazyk poskytuje možnosť definovať rozhrania pomocou kľúčového slova `interface`, ktoré upresňujú zamýšľaný tvar objektov. Kľúčové slovo `type` zas umožňuje vytváranie typových aliasov, či už primitívnych alebo komplexných typov. Okrem samotných typov podporuje typovú inferenciu čo pomáha s prehľadnosťou programu, keďže často nie je potrebné tvar objektu definovať manuálne.

1.3.3 React Flow

React Flow [1] je populárna knižnica na tvorbu grafových editorov, ktorá je súčasťou väčšieho projektu xyflow [3]. Zatiaľ čo React Flow je určený pre React, xyflow ponúka plnú kompatibilitu aj s niektorými inými podobnými knižnicami. Jej veľkou výhodou je široká škála možností prispôsobenia jednotlivých aspektov grafu, ako aj ich jednoduchá implementácia.

Každý vrchol predstavujú jeho atribúty a príslušný React komponent. Atribúty musia byť minimálne tvorené pozíciou a unikátnym identifikátorom. Ďalej však môžeme pridávať vlastné atribúty, ako aj komponenty, ktoré sa dajú prispôsobiť špecifickým potrebám našej aplikácie. Na veľmi podobnom princípe sa dajú konfigurovať aj hrany grafu, tie sú tiež tvorené komponentami a spolu s vrcholmi tvoria základnú štruktúru každého grafu.

Knižnica tiež poskytuje niektorú rozšírenú funkcionalitu akou je napríklad centrovanie grafov, uloženie a obnovenie stavu grafu alebo konfigurovateľný panel s nástrojmi.

1.3.4 Redux

Redux [2] je knižnica na správu stavu JavaScriptových aplikácií, často používaná spolu s knižnicou React na zjednodušenie tvorby komplexných používateľských rozhraní. Stav

aplikácie je reprezentovaný iba ako obyčajný objekt, ktorý predstavuje centrálne miesto pre ukladanie a čítanie dát, čím výrazne uľahčuje ich zdieľanie naprieč komponentami.

Stav v Reduxe nie je možné meniť priamo. Namiesto toho sa zmeny vykonávajú pomocou akcií a reducerov. Akcie sú objekty popisujúce konkrétnu udalosť v aplikácii. V prípade potreby môžu obsahovať aj doplňujúce údaje (payload). Tieto akcie ďalej spracúvajú reducery. To sú funkcie, ktoré na základe aktuálneho stavu a prijatej akcie určia nový stav aplikácie. Pre ich správne fungovanie je dôležité dbať na to, aby reducer pre rovnaký stav a rovnakú akciu vždy vrátil rovnaký výsledok, nemali by sa preto používať napríklad na generovanie náhodných hodnôt. Aplikácia môže obsahovať viacero reducerov, čo umožňuje rozdeliť logiku na menšie, prehľadné a nezávislé časti.

Na získavanie konkrétnych dát zo stavu sa používajú selektory. Ide o funkcie, ktoré na základe súčasného globálneho stavu vyberajú alebo odvodzujú potrebné údaje, čím zjednodušujú a sprehľadňujú prácu s dátami.

Kapitola 2

Súvisiace práce

V tejto kapitole sa podrobnejšie venujeme aplikáciám Prieskumník štruktúr a Prieskumník grafových štruktúr, ktoré sú pre túto prácu kľúčové. Postupne vysvetlíme, aký problém sa snažia vyriešiť, ako fungujú, a aké majú nedostatky. V závere kapitoly stručne popíšeme aplikáciu Logický pracovný zošit, do ktorej sú spomínané aplikácie integrované.

2.1 Prieskumník štruktúr

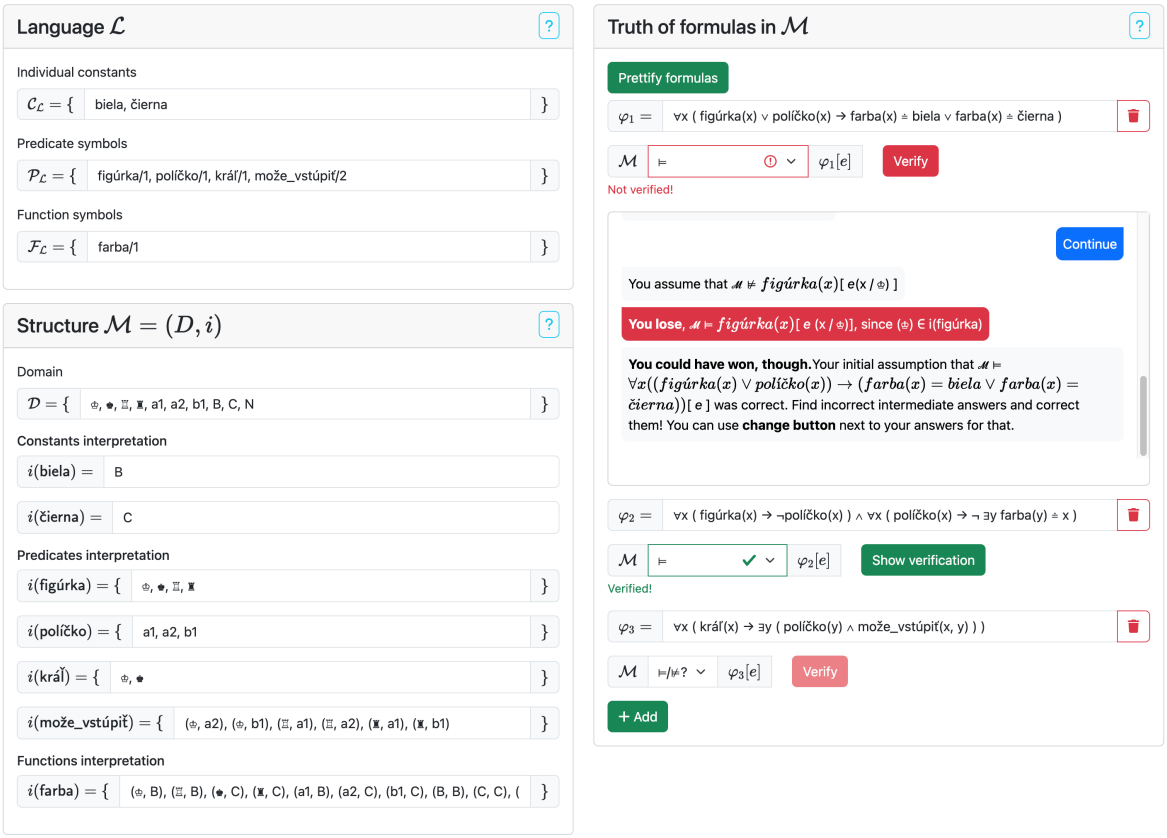
Prieskumník štruktúr je jednou z webových aplikácií používaných na predmete Logika pre informatikov. Jej primárnou úlohou je umožniť študentom interaktívne vytvárať vlastný jazyk a štruktúru logiky prvého rádu. V takto definovanej štruktúre je ďalej možná tvorba a analýza pravdivosti formúl s rýchlou spätnou väzbou.

Pôvodná aplikácia vznikla v rámci viacerých bakalárskych prác. Konkrétne sa jedná o prácu Milana Cifru [5], ktorá sa zaoberá samotným vývojom prieskumníka štruktúr, prácu Richarda Tótha [12], rozširujúcu prieskumník o Henkinovu-Hintikkovu hru a prácu Miroslava Baluchu [4], ktorá priniesla alternatívny grafový pohľad. Túto poslednú prácu ešte bližšie rozoberieme v nasledujúcej podkapitole 2.2.

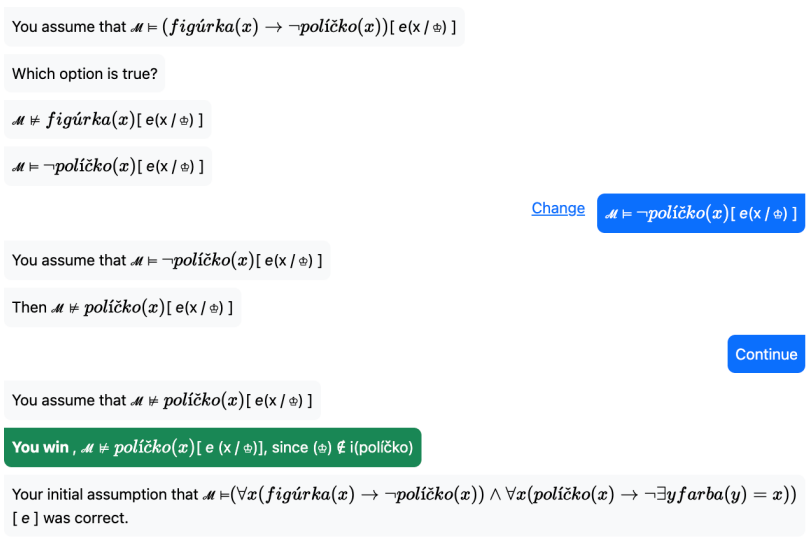
Implementácia aplikácia sa však časom stala neaktuálnou a nejednotnou, keďže bola výsledkom viacerých bakalárskych prác využívajúcich rôzne prístupy k použitým technológiám. To spôsobilo jej ťažkú udržateľnosť, ako aj náročnosť rozširovania o ďalšiu funkcionalitu.

Cieľom bakalárskej práce Jozefa Filipa [6] bolo vytvorenie novej, jednotnej verzie pôvodnej aplikácie bez grafového pohľadu a s novým spôsobom využitia Henkinovej-Hintikkovej hry. Túto verziu možno vidieť na obrázku 2.1.

V ľavej hornej časti obrázka sa nachádzajú textové vstupy na definovanie mimologický symbolov. V prípade predikátových a funkčných symbolov sa definuje aj ich arita vo formáte *symbol/arita*. Pod definíciou jazyka logiky prvého rádu sa definuje



Obr. 2.1: Používateľské rozhranie reimplentovanej verzie Prieskumníka štruktúr.



Obr. 2.2: Príklad dialógu Henkinovej-Hintikkovej hry.

samotná štruktúra, to vyžaduje zadanie domény a určenie interpretácie jednotlivých mimologických symbolov. Aplikácia poskytuje iba jeden, tzv. množinový pohľad na interpretáciu predikátových a funkčných symbolov, ktorý je opäť realizovaný pomocou textového vstupu.

V pravej časti aplikácie používateľ definuje formuly. Po vytvorení validného jazyka a štruktúry môže skúmať pravdivosť týchto formúl v danej štruktúre. Najprv však musí sám odhadnúť a zvoliť, či je formula pravdivá alebo nie. Svoj odhad si následne overí prostredníctvom Henkinovej-Hintikkovej hry. Tá funguje na princípe dialógu, v ktorom používateľ postupne dostáva otázky o pravdivosti čoraz menších podformúl pôvodnej formuly. Keď je už podformula dostatočne jednoduchá, hra používateľovi odhalí správnosť jeho predpokladu spolu s vysvetlením, prečo bol alebo nebol správny. Príklad dialógu je zobrazený na obrázku 2.2.

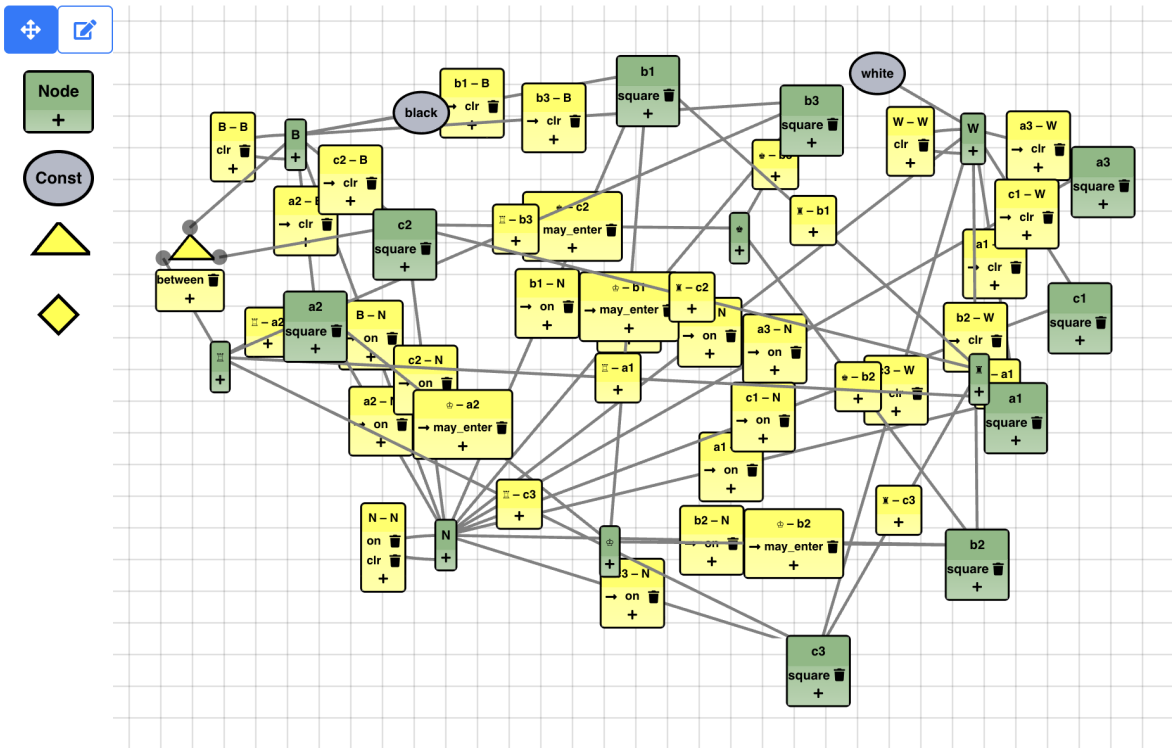
Asi najväčším problémom novej aplikácie v porovnaní s predošlou je nedostatok spôsobov vizualizácie a vytvárania interpretácií predikátových a funkčných symbolov. Množinová, resp. textová reprezentácia sa už pri pomerne malých interpretáciách stáva neprehľadnou a ťažko editovateľnou, keďže vyžaduje dodržiavanie správnej syntaxe pri zápise. Predošlá verzia riešila tento problém pridaním dvoch ďalších alternatívnych pohľadov. Konkrétne ide o tzv. maticový a databázový pohľad, v ktorých bolo možné pomocou zaškrtávania alebo výberu zo zoznamu určiť, ktoré prvky sa majú v interpretácií vyskytnúť.

2.2 Prieskumník grafových štruktúr

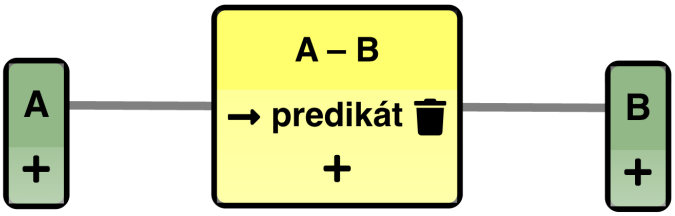
Úlohou bakalárskej práce Miroslava Baluchu [4] bolo rozšíriť aplikáciu Prieskumník štruktúr o nový pohľad na štruktúry logiky prvého rádu vo forme grafového editora. V jednom grafe sa mu podarilo pomocou rôznych typov vrcholov znázorniť konštanty, prvky domény a za pomoci hrán aj vzťahy medzi nimi. Príklad konkrétnej štruktúry je možné vidieť na obrázku 2.3.

S grafom možno interagovať v dvoch režimoch. Prvým je pohybový režim, ktorý dovoľuje iba mazanie a zmenu pozície jednotlivých objektov. V druhom, editovacom režime sú už sprístupnené funkcie ako vytváranie predikátových a funkčných symbolov, tvorba nových vzťahov alebo premenovávanie prvkov domény. Tlačidlá, pomocou ktorých sa medzi režimami prepína, sú viditeľné v ľavej hornej časti obrázka. Pod týmito tlačidlami sú umiestnené jednotlivé typy vrcholov. Ich potiahnutím do priestoru grafu sa vytvorí nová inštancia príslušného typu. Zelené vrcholy reprezentujú prvky domény a sivé zas konštanty. Žlté vrcholy pomáhajú pri tvorbe ternárnych a kvaternárnych vzťahov.

Binárny vzťah medzi prvkami domény je znázornený pomocou hrany spolu s jej



Obr. 2.3: Grafový editor v predošlej implementácii Prieskumníka štruktúr



Obr. 2.4: Príklad znázornenia binárneho vzťahu.

popisom (viď obrázok 2.4). Do popisu možno pridávať alebo odoberať predikátové a funkčné symboly, ktoré majú obsahovať tento vzťah vo svojej interpretácii. Pomocou šípky pri názve predikátu sa určuje smer vzťahu. Pri ternárnych a kvaternárnych vzťahoch je nutné najprv spojiť všetky prvky domény s vhodným vrcholom, pri ktorom sa následne dá meniť popis už spomínaným spôsobom.

Ako sa však časom ukázalo, implementácia mala niekoľko fundamentálnych problémov. Hlavným z nich bola snaha nahradiť takmer celú funkcionalitu prieskumníka štruktúr, čo na viacerých miestach viedlo ku kompromisom znižujúcich prehľadnosť aj intuitívnosť rozhrania. Azda najväčším vinníkom zlej prehľadnosti, resp. čitateľnosti, je spôsob zobrazovania binárnych vzťahov. Tých sa v štruktúrach používaných na predmete vyskytuje pomerne veľa, čo v kombinácií s ich rozsiahlou vizualizáciou vedie k prekrývaniu ostatných prvkov a výraznému zníženiu schopnosti orientácie sa v zobrazení.

Návrh niektorých ovládacích prvkov tiež pôsobil vo výsledku ťažkopádne, ako napríklad oddelenie editovania a presúvania do dvoch samostatných režimov. Ďalším problémom je nedostatok zaujímavých a unikátnych funkcií v porovnaní s textovými vstupmi, ktoré by ho spravili pre študentov atraktívnejším. Spomínané nedostatky sa prejavili aj na jeho nízkej popularite na predmete, kde sa prakticky vôbec nevyužíval.

2.3 Logický pracovný zošit

Vývojom tejto webovej aplikácie sa vo svojej bakalárskej práci zaoberal Matej Mok [11]. Pred jej vznikom boli nástroje na výučbu matematickej logiky dostupné iba v podobe samostatných aplikácií, čo sa však s ich rastúcim počtom stávalo nepraktickým. Logický pracovný zošit tieto nástroje integruje do jedného celku, čím učiteľom predmetu umožňuje vytvárať ucelenejšie cvičenia a žiakom uľahčuje ich vypracovávanie, ako aj odovzdávanie. Ďalšou výhodou pracovného zošita je priebežné ukladanie práce žiakov, vďaka čomu sa k nej môžu kedykoľvek vrátiť. Učitelia majú zároveň k uloženým prácam prístup, čo im dovoľuje ich prezerať a pridávať komentáre.

Kapitola 3

Návrh

TODO

3.1 Požiadavky

3.1.1 Zámer rozšírenia aplikácie!

Hlavným cieľom tejto práce je zlepšiť použiteľnosť existujúcej aplikácie Prieskumník štruktúr používanej na predmete Logika pre informatikov. Ako bolo už bližšie spomenuté v podkapitole venovanej práve tomuto nástroju 2.1, súčasná verzia poskytuje iba veľmi obmedzený spôsob vizualizácie a manipulácie s interpretáciami predikátových a funkčných symbolov.

Primárnym zlepšením v tejto oblasti má byť rozšírenie nástroja o grafové editory, ktoré používateľom pomôžu pri vizualizácií a vytváraní binárnych predikátov a unárnych funkcií logiky prvého rádu. Konkrétne sa jedná o Orientovaný graf, Bipartitný graf a Hasseho diagram. Každý z nich by mal z hľadiska funkčnosti slúžiť ako plnohodnotná alternatíva k už existujúcemu množinovému pohľadu. Musia teda zobrazovať jednotlivé usporiadané dvojice, resp. vzťahy, ktoré konkrétna interpretácia popisuje a prvky domény, medzi ktorými tieto vzťahy vznikajú.

Oproti predošlému riešeniu tiež prinesú vylepšenia na sprehľadnenie aj iných vzťahov v štruktúre. Konkrétne umožnia vizualizáciu unárnych predikátov, keďže tie často vedia používateľovi poskytnúť nápomocnú informáciu o vlastnostiach jednotlivých prvkov. Taktiež zobrazia, ktorými konštantami sú jednotlivé prvky domény označené. Podľa príslušnosti prvkov domény do interpretácií unárnych predikátov sa majú dať prvky aj filtrovať. Teda používateľ si musí byť schopný zvoliť, ktoré unárne predikáty chce zobraziť a prvky domény, ktoré sa nevyskytujú v žiadnej z ich interpretácií by sa mali z grafu skryť.

3.1.2 Ďalšie požiadavky

Počas vývoja sa požiadavky postupne menili a rozširovali. Jednou z pridaných požiadaviek je reimplementácia Maticového a Databázového editora z predošlej verzie Prieskumníka štruktúr. Ich cieľ je podobný ako pri grafových reprezentáciách, no okrem poskytnutia ďalšieho alternatívneho pohľadu sú na rozdiel od nich schopné reprezentovať aj iné ako binárne predikáty a unárne funkcie. Aj pri týchto editoroch sa požaduje aby bolo v nich možné unárne predikáty zobraziť, ako aj pomocou nich filtrovať.

Ďalším vylepšením má byť pridanie editora interpretácií funkcií.

3.2 Grafové editory

Tento návrh sa zameriava na tri zvolené grafové reprezentácie, konkrétne sa jedná o orientovaný graf, bipartitný graf a Hasseho diagram. Každá z nich bola zvolená pre svoje unikátne vlastnosti vhodné na skúmanie a konštrukciu binárnych predikátov logiky prvého rádu. V jednotlivých podkapitolách je podrobnejšie vysvetlená motivácia za ich výberom spolu s ich prvotným návrhom.

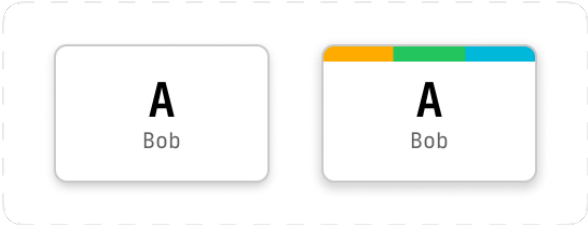
3.2.1 Orientovaný graf

Cieľom tejto grafovej reprezentácie je poskytnúť užívateľovi jednoduchý a prehľadný spôsob vizualizácie binárneho predikátu v logike prvého rádu.

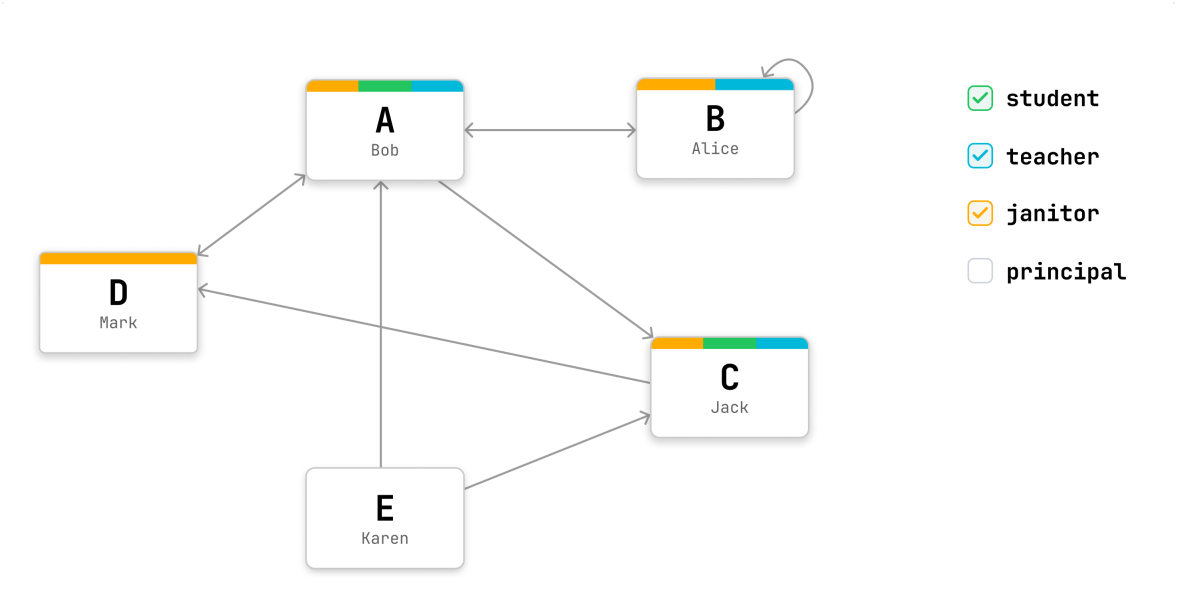
Od grafu sa požaduje, aby zobrazoval jednotlivé usporiadané dvojice, resp. vzťahy, ktoré interpretácia daného binárneho predikátu popisuje, prvky domény, medzi ktorými tieto vzťahy vznikajú, ako aj prípadné konštanty, ktoré sú týmito prvkami označené. Dôležité je tiež umožniť vizualizáciu unárnych predikátov, keďže tie často vedia užívateľovi poskytnúť nápomocnú informáciu o vlastnostiach jednotlivých prvkov.

V samotnom grafe prvky domény prirodzene predstavujú vrcholy, vzťahy sú zas reprezentované pomocou hrán. Konštanty a unárne predikáty som sa taktiež rozhodol zakomponovať ako súčasť vrcholov, keďže sú to informácie prislúchajúce prvkom domény. Výzva, ktorú takéto riešenie však prináša, je zachovať rovnováhu medzi množstvom informácií obsiahnutých vo vrcholoch a ich prehľadnosťou.

Prístup, ktorý som pri návrhu vrcholov zvolil, možno vidieť na obrázku 3.1. Najväčší typografický dôraz sa dáva na samotný prvok domény (na obrázku veľké písmeno A). Pod ním sa nachádzajú konštanty reprezentujúce tento prvok (na obrázku nápis Bob). Na vrchole vpravo zas možno vidieť panel predstavujúci unárne predikáty, do interpretácie ktorých daný prvok patrí, kde každá farba symbolizuje konkrétny predikát. Práve použitie farieb namiesto iných alternatív, ako napríklad nápisov s názvom unárneho predikátu, má pomôcť s celkovou prehľadnosťou vrcholov.



Obr. 3.1: Prvý návrh vizuálu vrcholov.



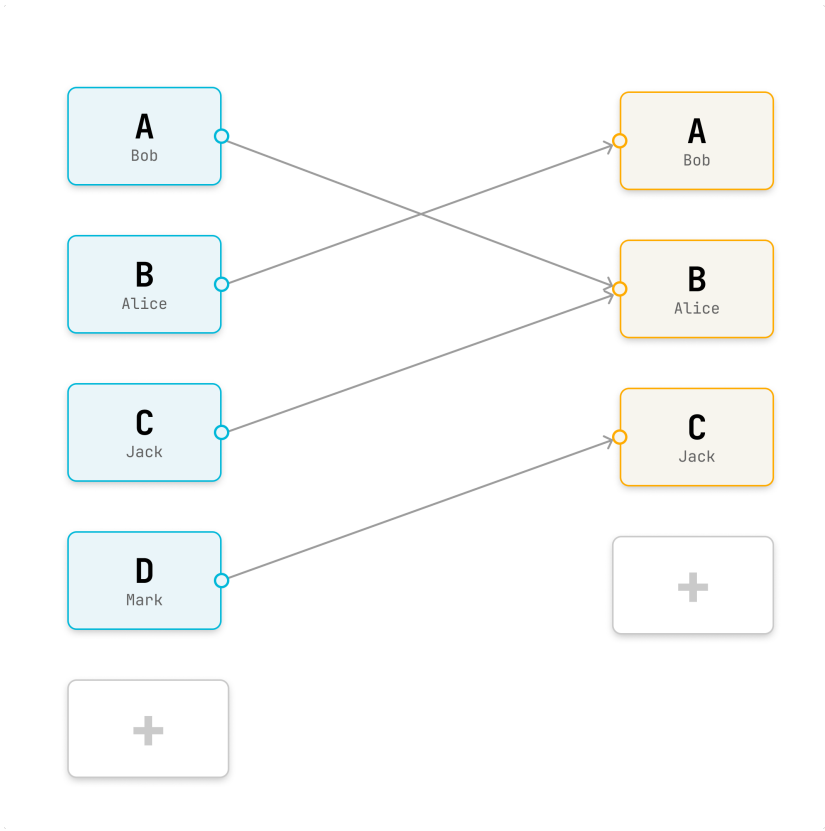
Obr. 3.2: Prvý návrh vizuálu orientovaného grafu.

Použitie týchto vrcholov na grafovú reprezentáciu konkrétneho binárneho predikátu znázorňuje obrázok 3.2. Hrany sú reprezentované šípkami, kde smer šípky upresňuje vzťah medzi prvkami. V pravej časti obrázka sa nachádza zoznam unárnych predikátov spolu s ich príslušnými farbami. Medzi predikátmi, ktoré sa majú zobraziť, je užívateľovi umožnené filtrovať zaškrtnutím k nemu prislúchajúceho políčka.

3.2.2 Bipartitný graf

Zatiaľ čo orientovaný graf je vhodný na získanie všeobecného prehľadu o vzťahoch vyskytujúcich sa v binárnych predikátoch, bipartitný graf má potenciál poskytnúť lepší náhľad na viac špecifické vlastnosti. Napríklad v kontexte zobrazení to môžu byť vlastnosti ako surjektívnosť alebo bijektívnosť. Všeobecne sa však nemusíme obmedzovať len na zobrazenia. Na dosiahnutie zaujímavých výsledkov stačí, aby sa skúmané prvky dali vhodne rozdeliť do dvoch zmysluplných skupín. Úlohou pri tejto grafovej reprezentácii je teda prispôsobiť rozhranie grafu skúmaniu takýchto vlastností.

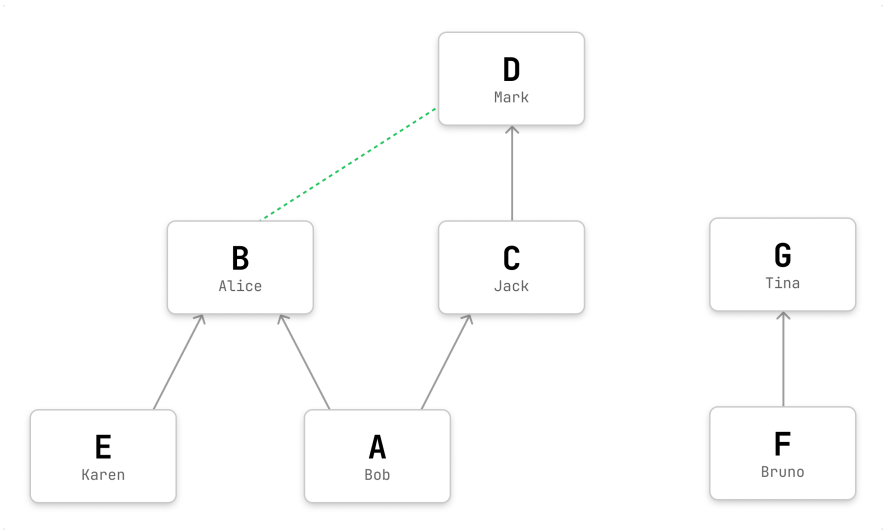
Základné prvky grafu, ako vrcholy a hrany, sú v zásade zhodné s tými z predošlej reprezentácie. V tomto prípade sa ale vyžaduje, aby boli prvky domény rozdelené do



Obr. 3.3: Prvý návrh vizuálu pre bipartitný graf.

dvoch vizuálne odlíšiteľných celkov (neznamená to však, že musia byť disjunktné). Ďalším rozdielom je spôsob vytvárania hrán. Tie sa dajú vždy vytvárať iba z jednej, predom určenej skupiny, nikdy nie z oboch. Taktiež sa hrany nedajú vytvárať medzi prvkami v rovnakej skupine. Práve tieto obmedzenia robia graf vhodnejší na skúmanie vlastností zobrazení, keďže jeden celok si môžeme predstaviť ako definičný obor (z neho hrany vytvárame) a druhý ako obor hodnôt.

Mnou navrhnuté riešenie možno vidieť na obrázku 3.3. Hrany reprezentujú šípky smerujúce zľava doprava. Vrcholy sú rozdelené do dvoch stĺpcov, čo je naznačené aj ich rozdielnou farbou. Každý stĺpec reprezentuje ľubovoľnú podmnožinu prvkov danej domény. Užívateľovi je tiež umožnené meniť poradie vrcholov, ale iba v rámci jeho skupiny, presúvanie medzi nimi nie je dovolené. Vynechaný je panel predstavujúci unárne predikáty, keďže je nateraz otázne, aký veľký by bol jeho prínos pri skúmaní vzťahov špecifických pre bipartitný graf. Posledný, menej výrazný vrchol je iný od ostatných, pretože slúži ako tlačidlo na pridávanie nových prvkov do príslušného stĺpca. Spôsob pridávania vrcholov by mal byť implementovaný ako zoznam, resp. filter, doteraz nepridaných prvok, z ktorých si bude môcť užívateľ vybrať.



Obr. 3.4: Prvý návrh vizuálu pre Hasseho diagram.

3.2.3 Hasseho diagram

Výhodou Hasseho diagramu je jeho schopnosť výrazne zjednodušiť a sprehľadniť prácu s čiastočnými usporiadaniami. Samotný orientovaný graf nie je ideálny pre tento účel z viacerých dôvodov. Napríklad pri čiastočných usporiadaniach s väčším počtom prvkov sa stáva neprehľadným, keďže vždy zobrazuje všetky hrany, čo často nie je nutné. Ďalším je, že aj jednoduchá úprava takéhoto čiastočného usporiadania môže v orientovanom grafe znamenať potrebu ručne vytvoriť množstvo nových hrán, kvôli nutnosti zachovania tranzitivity, čím sa uberá na použiteľnosti. Cieľom je navrhnúť vhodnú grafovú reprezentáciu Hasseho diagramu, ktorá (ako už bolo spomínané v podkapitole 1.2) dokáže s týmito problémami pomôcť.

Vrcholy a hrany grafu ostanú opäť výrazne nezmenené, no kvôli prehľadnosti bude zásadné dodržať správnu vertikálnu hierarchiu vrcholov, ktorá je tiež súčasťou konštrukcie Hasseho diagramu. Zámerom však nie je užívateľa akokoľvek obmedzovať za účelom dodržiavania tejto hierarchie, ako tomu bolo v bipartitnom grafe, ale poskytnúť mu dobrý základ rozmiestnenia vrcholov, ktorý je ďalej prispôsobiteľný vlastným predstavám. Dôležitá bude tiež korektná implementácia zvyšných vlastností Hasseho diagramu, ako aj validácia nových hrán pridaných užívateľom. Dôraz na validáciu je kladený hlavne kvôli tomu, že pridaním nesprávnej hrany môže reprezentácia prestať dávať zmysel.

Môj návrh možno vidieť na obrázku 3.4. Viditeľná je jasná hierarchia vrcholov s orientovanými hranami smerujúcimi nahor. Zelenou prerušovanou čiarou je užívateľovi naznačená korektnosť hrany, ktorú sa práve snaží pridať (tu je hrana v poriadku). V opačnom prípade by hrana bola červená a pokus o jej vytvorenie by nemal žiadny účinok. Ostatné prvky sú podobné ako v predošlých reprezentáciách.

3.3 Tabuľkové editory

TODO

3.3.1 Maticový editor

TODO

3.3.2 Databázový editor

TODO

3.4 Editor interpretácií funkcií

TODO

3.5 Spoločné rozhranie editorov

TODO

3.6 Nový stav aplikácie

TODO

3.7 Queries

Kapitola 4

Implementácia

TODO

4.1 Komponent editorov

TODO

4.2 Stav aplikácie

TODO

4.2.1 Štruktúra

TODO

4.2.2 Synchronizácia

TODO

4.3 Integrácia s Logickým pracovným zošitom

TODO

4.3.1 Importovanie a exportovanie stavu

TODO

4.3.2 Logický kontext

TODO

4.4 Ďalšie vylepšenia Prieskumníka štruktúr

TODO

4.4.1 Queries

TODO

4.4.2 Refaktorizácia Henkinovej-Hintikkovej hry

TODO

Kapitola 5

Testovanie ?

TODO

Záver

Podarilo sa nám jasne stanoviť ciele a požiadavky na nové grafové editory pre prieskumník štruktúr logiky prvého rádu. Jednotlivé grafové reprezentácie sa zatiaľ ukázali byť vhodné a dobre sa navzájom dopĺňajúce na dosiahnutie zmysluplných, intuitívnych a vysoko informatívnych vizualizačných prostriedkov pre študentov.

Ďalej sme sa v prvom návrhu zaoberali nápadmi, ako premeniť tieto reprezentácie do interaktívnej podoby a ešte lepšie využiť ich potenciál s ohľadom na naše špecifické požiadavky. Nakoniec sa ukázala byť prínosná aj pilotná implementácia, ktorá v mnohých ohľadoch utvrdila správnosť myšlienok predstavených v predchádzajúcich kapitolách, no zároveň odhalila nedostatky niektorých riešení a potrebu ich vylepšenia v ďalších iteráciách.

Pri mojom vlastnom testovaní sa prejavila najmä slabá intuitívnosť častých úkonov, ako napríklad pridávania alebo mazania nových vrcholov grafov. K tomuto problému tiež prispieva fakt, že na vzor prvotného návrhu je táto funkcionalita implementovaná pre bipartitný graf inak, ako pre ostatné dve reprezentácie. Tento problém by som chcel vyriešiť kompletnou zmenou a zjednotením spôsobu ich pridávania, napríklad implementáciou mechanizmu filtrovania vrcholov, podobnému tomu pri volení unárnych predikátov v orientovanom grafe. V niektorých aspektoch je zas zrejmé, že vizuálna podoba pilotnej implementácie sa dostatočne nedrží prvotného návrhu, na čom bude ešte tiež potrebné zapracovať.

Medzi menej viditeľné, no o nič menej zásadné problémy patrí nutnosť doladiť spôsob, akým jednotlivé grafy spolu komunikujú zmeny týkajúce sa ich spoločného stavu. Pri tejto miere interaktivity, ktorú knižnica React umožňuje, je dôležité dbať aj na efektívnosť implementovaných riešení. V opačnom prípade to môže viesť k zlému zážitku pri interakcii s aplikáciou, z dôvodu neresponzívneho používateľského rozhrania. Tento aspekt bude predstavovať jeden z mojich hlavných cieľov.

Literatúra

[1] *React Flow: A library for building node-based UIs*, 2026. Dostupné z `https://reactflow.dev`.

[2] *Redux: A JS library for predictable and maintainable global state management*, 2026. Dostupné z `https://redux.js.org`.

[3] *XFlow: React libraries for node-based UIs*, 2026. Dostupné z `https://xyflow.dev`.

[4] Miroslav Baluch. Prieskumník grafových štruktúr pre logiku prvého rádu. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2020.

[5] Milan Cifra. Prieskumník sémantiky logiky prvého rádu. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2018.

[6] Jozef Filip. Reimplementácia prieskumníka štruktúr pre logiku prvého rádu. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2025.

[7] R.P. Grimaldi. *Discrete and combinatorial mathematics: an applied introduction.* Addison-Wesley Pub. Co., 1985.

[8] Ján Kľuka, Ján Mazák, and Jozef Šiška. Logika pre informatikov a Úvod do matematickej logiky: Poznámky z prednášok. 2025.

[9] Meta Platforms, Inc. *React: A JavaScript library for building user interfaces*, 2026. Dostupné z `https://react.dev`.

[10] Microsoft. *TypeScript: Typed JavaScript at Any Scale*, 2026. Dostupné z `https://www.typescriptlang.org`.

[11] Matej Mok. Interaktívny pracovný zošit pre výučbu logiky pre informatikov. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2022.

[12] Richard Tóth. Henkinova-hintikkova hra v prieskumníku štruktúr. Bakalárska práca, Univerzita Komenského v Bratislave, Fakulta matematiky, fyzika a informatiky, 2021.